DataLink Engine — Complete REST API Specification

Interactive Endpoints, Pydantic Schemas, Filtering & Streaming Export Reference

OpenAPI 3.0 | RESTful | FastAPI | Pydantic v2 | Excel & CSV Streaming | Production Ready

API Version: `v1`
Base URL (Production / Vercel Proxy): `https://prototype-azure-theta.vercel.app/api`
Base URL (Direct Railway Backend): `https://web-production-9dadf.up.railway.app/api`
Base URL (Local Development / VS Code): `http://127.0.0.1:8001/api`
Interactive Swagger / OpenAPI UI: `http://127.0.0.1:8001/docs` or `https://web-production-9dadf.up.railway.app/docs`

---

1. Overview & Authentication

The DataLink API is a high-performance RESTful service built with FastAPI and Python 3.12. All data transfer uses UTF-8 JSON payloads, except for file upload endpoints which consume `multipart/form-data` and export endpoints which stream binary `.xlsx` and text `.csv`.

---

2. Endpoints Summary Table

CATEGORY	HTTP METHOD	PATH	DESCRIPTION
Upload & Inspection	`POST`	`/upload/inspect`	Inspect workbook sheets and preview column mappings without persisting.
	`POST`	`/upload` (or `/files/upload`)	Upload an Excel or CSV file to create a processing job.
Job Execution	`POST`	`/jobs/{job_id}/mapping-overrides`	Apply custom column remappings to a job.
	`POST`	`/jobs/{job_id}/start`	Trigger background processing for an uploaded job.
	`GET`	`/jobs`	Paginated list of all job execution runs.
	`GET`	`/jobs/{job_id}`	Real-time status, progress %, and metrics of a specific job.
	`GET`	`/jobs/{job_id}/errors`	Row-level validation failure audit trail for a job.
Records & Retrieval	`GET`	`/records`	Search, filter, and paginate normalized real estate records.
	`GET`	`/records/{record_id}`	Retrieve full details of a single record.
	`PUT`	`/records/{record_id}`	Update fields of an existing record.
	`GET`	`/records/filters`	Distinct values for all frontend filter dropdowns.
	`GET`	`/records/export`	Stream filtered dataset to **Excel (`.xlsx`)** or **CSV (`.csv`)**.
Analytics & Schema	`GET`	`/dashboard/stats`	Aggregated metrics, completeness %, and community breakdown.
	`GET`	`/column-mappings`	Catalog of 23 target canonical fields with recognized aliases.

3. Detailed Endpoint Specifications

3.1 Upload & File Inspection

`POST /api/upload/inspect`

Inspects an Excel/CSV file in memory, extracts sheet names, estimates total rows, and suggests canonical field mappings.

- **Content-Type**: `multipart/form-data`
- **Form Field**: `file` (Binary File: `.xlsx`, `.xls`, `.csv`)
- **Response** `200 OK`:

```
{
  "filename": "Club Villas.xlsx",
  "detected_format": "openpyxl",
  "total_rows_estimate": 142,
  "header_count": 16,
  "mapped_count": 14,
  "mapped_columns_preview": [
    { "raw_header": "Owner Name", "mapped_target": "Name" },
    { "raw_header": "Mobile", "mapped_target": "Mobile 1" },
    { "raw_header": "Villa No", "mapped_target": "Unit Number" },
    { "raw_header": "Community", "mapped_target": "Community" }
  ],
  "sheets": [
    { "name": "Sheet1", "total_rows": 142, "n_cols": 16, "is_reference": false }
  ]
}
```

`POST /api/upload`

Uploads and persists a source file into the system, generating a unique `job_id`.

- **Content-Type**: `multipart/form-data`
- **Query Parameters**:
- `batch_size` *(integer, optional, default: 1000)*: Rows processed per database transaction chunk.
- `autostart` *(boolean, optional, default: false)*: Immediately begin execution without waiting for mapping confirmation.
- `force` *(boolean, optional, default: false)*: Re-process even if content SHA-256 matches a prior run.
- **Response** `201 Created`:

```
{
  "job_id": 42,
  "source_file_id": 18,
  "filename": "Club Villas.xlsx",
  "size_bytes": 27480,
  "status": "UPLOADED",
  "created_at": "2026-08-21T10:00:00Z"
}
```

3.2 Job Execution & Audit

`POST /api/jobs/{job_id}/start`

Starts the in-process batch pipeline for the specified job.

- **Response** `200 OK`:

```
{
  "job_id": 42,
  "status": "READING",
  "message": "Processing started in background."
}
```

`GET /api/jobs/{job_id}`

Polls the real-time execution status of a running or completed job.

- **Response** `200 OK`:

```
{
  "id": 42,
  "source_file_id": 18,
  "filename": "Club Villas.xlsx",
  "status": "COMPLETED",
  "total_rows": 142,
  "processed_rows": 142,
  "valid_rows": 138,
  "invalid_rows": 0,
  "duplicate_rows": 4,
  "skipped_rows": 0,
  "error_count": 0,
  "progress_percent": 100.0,
  "current_sheet": null,
  "started_at": "2026-08-21T10:00:01Z",
  "finished_at": "2026-08-21T10:00:04Z"
}
```

3.3 Records Search, Retrieval & Export

`GET /api/records`

Search and filter normalized real estate records.

- **Query Parameters**:
- `q` *(string, optional)*: Free-text search across Name, Community, Unit, Mobile, Developer, Plot.
- `community` *(string, optional)*: Filter by exact community name (e.g. `Dubai Hills`).
- `property_type` *(string, optional)*: Filter by type (`Residential`, `Commercial`, `Land`).
- `bedroom` *(string, optional)*: Filter by bedroom count (`Studio`, `1 BR`, `2 BR`, `3 BR`, etc.).
- `developer` *(string, optional)*: Filter by developer name (e.g. `Emaar Properties`).
- `status` *(string, optional)*: Record status (`VALID` [default], `INCOMPLETE`, `DUPLICATE`, `INVALID`, `ALL_WITH_INCOMPLETE`).
- `sort_by` *(string, default: "id")*: Sort column (`name`, `community`, `unit_number`, `developer`, `procedure_value`, etc.).
- `sort_dir` *(string, default: "desc")*: Direction (`asc` or `desc`).
- `page` *(integer, default: 1)*: Page number.
- `page_size` *(integer, default: 50, max: 500)*: Records per page.
- **Response** `200 OK`:


```
{
  "items": [
    {
      "id": 101,
      "name": "MINTESH HITESHBHAI SHAH",
      "community": "Dubai Hills",
      "sub_community": "MULBERRY AT PARK HEIGHTS",
      "building_cluster": "2",
      "unit_number": "G08",
      "bedroom": "2 BR",
      "property_type": "Residential",
      "developer": "Emaar Properties",
      "procedure_value": 2450000.0,
      "size": 1380.5,
      "mobile_1": "+971555064172",
      "mobile_2": null,
      "email_address": null,
      "status": "VALID",
      "source_file": "MULBERRY at PARK HEIGHTS.xlsx"
    }
  ],
  "total": 18450,
  "page": 1,
  "page_size": 50,
  "total_pages": 369,
  "has_next": true,
  "has_prev": false
}
```

`GET /api/records/export`

Exports the dataset matching the active search and filter query. Streams either an Excel spreadsheet or CSV file.

- **Query Parameters**: Same parameters as `/api/records`, plus:
- `format` *(string, required)*: `"xlsx"` or `"csv"`.
- `limit` *(integer, default: 50000, max: 100000)*: Max records to export.
- **Response** `200 OK`:
- If `format=xlsx`: `Content-Type: application/vnd.openxmlformats-officedocument.spreadsheetml.sheet` (Formatted with dark navy headers, bold text, frozen top row).
- If `format=csv`: `Content-Type: text/csv; charset=utf-8` (Encoded with `utf-8-sig` BOM for native Excel compatibility).
- Header: `Content-Disposition: attachment; filename="datalink_records_export_20260821_110000.xlsx"`

4. cURL Usage Examples

1. Ingest a File (Upload & Start)

```
# Step 1: Upload
JOB_DATA=$(curl -s -X POST "http://127.0.0.1:8001/api/upload?batch_size=500" \
  -F "file=@Dubai_Hills_Batch1.xlsx")
JOB_ID=$(echo $JOB_DATA | python -c "import sys, json; print(json.load(sys.stdin)['job_id'])")

# Step 2: Start Processing
curl -X POST "http://127.0.0.1:8001/api/jobs/$JOB_ID/start"
```

2. Search Records by Developer & Property Type

```
curl "http://127.0.0.1:8001/api/records?developer=Emaar%20Properties&property_type=Residential&status=VAL
```

3. Download Filtered Dataset as Excel (.xlsx)

```
curl "http://127.0.0.1:8001/api/records/export?format=xlsx&community=Dubai%20Hills&status=VALID" \
  --output "Dubai_Hills_Outreach_List.xlsx"
```